Java 1.6 Enhancement in Collection Framework

08 June 2025 10:27

1. NavigableSet (I)

2. NavigableMap (I)

Collection (I) 1.2V

Set (I) 1.2V

SortedSet (I) 1.2V

NavigableSet (I) 1.6V

TreeSet (1.2)V ← class is re-engineered in 1.6V (again)

NavigableSet (I) : It is a child interface of SortedSet and it defines several methods for navigation purpose.

NavigableSet defines the following methods :-

floor(e) ← it returns highest element which is <= e.

lower(e) ← it returns highest element which is < e.

ceiling(e) ← it returns lowest element which is >= e.

higher(e) ← it returns lowest element which is > e.

pollFirst() ← remove and return first element.

pollLast() ← remove and return last element.

descendingSet() ← it returns NavigableSet in reverse order.

Methods from SortedSet:-

first(), last(), headSet, tailSet(), subset().

public interface NavigableSet<E> extends SortedSet<E>

A SortedSet extended with navigation methods reporting closest matches for given search targets. Methods lower(E), floor(E), ceiling(E), and higher(E) return elements respectively less than, less than or equal, greater than or equal, and greater than a given element, returning null if there is no such element.

A NavigableSet may be accessed and traversed in either ascending or descending order. The descendingSet() method returns a view of the set with the senses of all relational and directional methods inverted. The performance of ascending operations and views is likely to be faster than that of descending ones. This interface additionally defines methods pollFirst() and pollLast() that return and remove the lowest and highest element, if one exists, else returning null. Methods subSet(E, boolean, E, boolean), headSet(E, boolean), and tailSet(E, boolean) differ from the like-named SortedSet methods in accepting additional arguments describing whether lower and upper bounds are inclusive versus exclusive. Subsets of any NavigableSet must implement the

NavigableSet interface.
The return values of navigation methods may be ambiguous in implementations that permit null elements. However, even in this case the result can be disambiguated by checking contains(null). To avoid such issues, implementations of this interface are encouraged to *not* permit insertion of null elements. (Note that sorted sets of Comparable elements intrinsically do not permit null.)
Methods subSet(E, E), headSet(E), and tailSet(E) are specified to return SortedSet to allow existing implementations of SortedSet to be compatibly retrofitted to implement NavigableSet, but extensions and implementations of this interface are encouraged to override these methods to return NavigableSet.
This interface is a member of the Java Collections Framework.

Since:

1.6

From <https://docs.oracle.com/en/java/javase/24/docs/api//java.base/java/util/NavigableSet.html>
For methods click the link give above.

```java
package Chapter7;
import java.util.*;
public class NavigableSetDemo {
    public static void main(String[] args) {
        TreeSet<Integer> t = new TreeSet<Integer>();
        t.add(2000);
        t.add(4000);
        t.add(5000);
        t.add(3000);
        t.add(1000);
        t.add(1500);
        System.out.println(t);
        System.out.println(t.ceiling(2000));
        System.out.println(t.higher(2000));
        System.out.println(t.floor(3000));
        System.out.println(t.lower(3000));
        System.out.println(t.pollFirst());
        System.out.println(t.pollLast());
        System.out.println(t.descendingSet());
        System.out.println(t);
    }
}
```

>=
>
<=
<

Output:
[1000, 1500, 2000, 3000, 4000, 5000]
2000
3000
3000
2000
1000 element removed.
5000 "
[4000, 3000, 2000, 1500]
[1500, 2000, 3000, 4000]

actual TreeSet object.

NavigableMap interface

It is a child interface of SortedMap(I).
It defines several methods for navigational purposes.

Navigable (I) defines the following methods:

floorkey (e)
lowerkey(e)
ceilingKey(e)
higherkey(e)
pollFirtEntry()
pollLastEntry()
descendingMap().

Map(I) 1.2v
|
SortedMap (I) 1.2v
|
NavigableMap (I) 1.6v
|
TreeMap (1.2v).

hierarchy of interfaces.

public interface NavigableMap<K,V> extends SortedMap<K,V>

A SortedMap extended with navigation methods returning the closest matches for given search targets. Methods lowerEntry(K), floorEntry(K), ceilingEntry(K), and higherEntry(K) return Map.Entry objects associated with keys respectively less than, less than or equal, greater than or equal, and greater than a given key, returning null if there is no such key. Similarly, methods lowerKey(K), floorKey(K), ceilingKey(K), and higherKey(K) return only the associated keys. All of these methods are designed for locating, not traversing entries.

A NavigableMap may be accessed and traversed in either ascending or descending key order. The descendingMap() method returns a view of the map with the senses of all relational and directional methods inverted. The performance of ascending operations and views is likely to be faster than that of descending ones. Methods subMap(K, boolean, K, boolean), headMap(K, boolean), and tailMap(K, boolean) differ from the like-named SortedMap methods in accepting additional arguments describing whether lower and upper bounds are inclusive versus exclusive. Submaps of any NavigableMap must implement the NavigableMap interface.

This interface additionally defines methods firstEntry(), pollFirstEntry(), lastEntry(), and pollLastEntry() that return and/or remove the least and greatest mappings, if any exist, else returning null.

The methods ceilingEntry(K), firstEntry(), floorEntry(K), higherEntry(K), lastEntry(), lowerEntry(K), pollFirstEntry(), and pollLastEntry() return Map.Entry instances that represent snapshots of mappings as of the time of the call. They do *not* support mutation of the underlying map via the optional setValue method.

Methods subMap(K, K), headMap(K), and tailMap(K) are specified to return SortedMap to allow existing implementations of SortedMap to be compatibly retrofitted to implement NavigableMap, but extensions and implementations of this interface are encouraged to override these methods to return NavigableMap. Similarly, SortedMap.keySet() can be overridden to return NavigableSet.

This interface is a member of the Java Collections Framework.

Since:

1.6

From <https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/NavigableMap.html>

Method Summary

All Methods | Instance Methods | Abstract Methods | Default Methods

Modifier and Type	Method	Description
Map.Entry<K,V>	ceilingEntry(K key)	Returns a key-value mapping associated with the least key greater than or equal to the given key, or null if there is no such key.
K	ceilingKey(K key)	Returns the least key greater than or equal to the given key, or null if there is no such key.
NavigableSet<K>	descendingKeySet()	Returns a reverse order NavigableSet view of the keys contained in this map.
NavigableMap<K,V>	descendingMap()	Returns a reverse order view of the mappings contained in this map.
Map.Entry<K,V>	firstEntry()	Returns a key-value mapping associated with the least key in this map, or null if the map is empty.
Map.Entry<K,V>	floorEntry(K key)	Returns a key-value mapping associated with the greatest key less than or equal to the given key, or null if there is no such key.
K	floorKey(K key)	Returns the greatest key less than or equal to the given key, or null if there is no such key.
SortedMap<K,V>	headMap(K toKey)	Returns a view of the portion of this map whose keys are strictly less than toKey.
NavigableMap<K,V>	headMap(K toKey, boolean inclusive)	Returns a view of the portion of this map whose keys are less than (or equal to, if inclusive is true) toKey.

ableMap.html#descendingMap()

Map.Entry<K,V>	higherEntry(K key)	Returns a key-value mapping associated with the least key strictly greater than the given key, or null if there is no such key.
K	higherKey(K key)	Returns the least key strictly greater than the given key, or null if there is no such key.
Map.Entry<K,V>	lastEntry()	Returns a key-value mapping associated with the greatest key in this map, or null if the map is empty.
Map.Entry<K,V>	lowerEntry(K key)	Returns a key-value mapping associated with the greatest key strictly less than the given key, or null if there is no such key.
K	lowerKey(K key)	Returns the greatest key strictly less than the given key, or null if there is no such key.
NavigableSet<K>	navigableKeySet()	Returns a NavigableSet view of the keys contained in this map.
Map.Entry<K,V>	pollFirstEntry()	Removes and returns a key-value mapping associated with the least key in this map, or null if the map is empty (optional operation).
Map.Entry<K,V>	pollLastEntry()	Removes and returns a key-value mapping associated with the greatest key in this map, or null if the map is empty (optional operation).
default NavigableMap<K,V>	reversed()	Returns a reverse-ordered view of this map.
NavigableMap<K,V>	subMap(K fromKey, boolean fromInclusive, K toKey, boolean toInclusive)	Returns a view of the portion of this map whose keys range from fromKey to toKey.
SortedMap<K,V>	subMap(K fromKey, K toKey)	Returns a view of the portion of this map whose keys range from fromKey, inclusive, to toKey, exclusive.

SortedMap<K,V>	tailMap(K fromKey)	Returns a view of the portion of this map whose keys are greater than or equal to fromKey.
NavigableMap<K,V>	tailMap(K fromKey, boolean inclusive)	Returns a view of the portion of this map whose keys are greater than (or equal to, if inclusive is true) fromKey.

```java
import java.util.*;
public class NavigableMapDemo {
   public static void main(String[] args) {
      TreeMap<String, String> t = new TreeMap<String, String>();
      t.put("c", "Cat");
      t.put("b", "Banana");
      t.put("d", "Dog");
      t.put("a", "Apple");
      t.put("g", "Gun");
      System.out.println(t);
      System.out.println(t.ceilingKey("c"));
      System.out.println(t.higherKey("c"));
      System.out.println(t.floorKey("e"));
      System.out.println(t.lowerKey("e"));
      System.out.println(t.lowerKey("d"));
      System.out.println(t.pollFirstEntry());
      System.out.println(t.pollLastEntry());
      System.out.println(t.descendingMap());
      System.out.println(t);
   }
}
```

Output:

```
{a=Apple, b=Banana, c=Cat, d=Dog, g=Gun}
c
d
d
d
c
a=Apple
g=Gun
{d=Dog, c=Cat, b=Banana}
{b=Banana, c=Cat, d=Dog}
```